

Transaction & Expense Updates — Plan Before Implementation

Status: Plan only. Nothing in this document has been built yet.

Prepared by: Paul Reggie Valenzuela

Date: October 7, 2026

Inputs reviewed: `LIST - DAY END ADT.csv` (Marikina Day End Report, October 3, 2026), `LIST - Expenses.csv` (Operating Expenses, 11 rows), and the current code for the transaction form, procedures, payments, CSV import and expense form.

1. Summary

#	Request	Recommended solution	Size
1	PWD / discounted price	Per-line list price + discount (percent or peso amount) + reason. PWD is a preset.	M
2	Multiple procedures per transaction	New <code>invoices</code> header table; each procedure stays one row in <code>transactions</code> (so commission and dashboards keep working).	L
3	VAT / non-VAT per procedure	One <code>vat_type</code> per line; VAT amounts are computed , never typed. Default comes from the procedure.	S
4	Patient table, typeahead, new/returning	Yes, add a <code>patients</code> table. Server-side typeahead; new/returning computed automatically, with override.	M
5	Manual invoice number	Plain editable field following the booklet. Duplicate warning per branch. Auto-generated invoice number is removed.	S
6	Proof of payment upload	Private object storage, metadata in a <code>payment_proofs</code> table, plus a separate "reference no." field.	M (needs a hosting decision)
7	Add Expenses fields from the CSV	Extend <code>expenses</code> with supplier, TIN, reference no., nature, VAT type and source of fund; add a <code>suppliers</code> table with typeahead.	M
0	<i>(found while checking the CSV)</i>	The current CSV importer rejects the new Day End layout. Fix first.	S

Ten decisions are listed in section 6. Four of them change the design if answered differently, so please look at those before I start.

2. What the two CSVs tell us

2.1 Day End report

Layout: a title row (`MARIKINA DAY END REPORT` , date), header row, 9 data rows, then a `Breakdown` block by payment method and a `Total` row. The total of 3,965.00 reconciles with the sum of Amount Paid.

What the data shows, and why it matters for the plan:

- **Discounts are hiding inside the procedure text.** OP(Moderate) w/5% Discount Dental Network Member was paid 1,615.00. That is exactly 1,700.00 less 5%. The list price is lost, and because the app creates a new row in procedures for every distinct procedure string, this one becomes its own "procedure". Commission rates (set per procedure) will never match it.
- **Several procedures are typed into one cell.** Sanitation + Ortho Adj (1,300.00) and Sanitation + Ortho Adj Fully Paid (300.00). There is one amount, so we cannot tell what each procedure cost, who earns commission on which, or count them correctly in "Popular procedures". "Fully Paid" is a payment status typed into the procedure name.
- **Reference numbers live in Remarks.** Six of nine rows carry a GCash, GoTyme or GHL approval number there (for example ref no. 0045705066371, GHL CREDIT APPROVAL NO. 007294). That is the "proof of payment" today.
- **Invoice number is mostly blank.** Only 1 of 9 rows has one (4543). So the field cannot be made mandatory without blocking normal days.
- **VAT is zero on every row,** and Vat Exclusive equals Amount Paid on every row. Everything sold in this sample is non-VAT.
- **Card fees follow fixed rates.** On the 1,615.00 GHL credit payment: merchant fee 45.22 is exactly 2.80%, withholding tax 8.08 is 0.50% (8.075 rounded up). The sheet's net collection is 1,561.71; subtracting the two gives 1,561.70, so the sheet is one centavo off.
- **Reservation fees are in the totals.** Four GCash reservation fees (600.00) are part of the 3,965.00. This answers the open question in csv-import-requirements.md ("should Reservation types count toward sales totals?"). They have no dentist and a blank Net Collection.
- **A zero-amount visit exists** (Follow up Check Up - no payment). The form and importer must keep allowing 0.00.
- **Names are consistently Last, First** (Zamora, Caroline). That makes prefix typeahead and duplicate detection practical.
- **Visit IDs follow two patterns:** 202610-003-001 for visits and RES20261003-010 for reservations. The app currently generates 20260921-001 .
- **The new layout has no Transaction Type, Visit Type, Month or Year columns.** The older sample did.

2.2 Expenses report

Header row plus 11 rows. The invoice amounts add up to 123,891.61, which matches the header total.

CSV column	What it holds	Evidence
Date	MM/DD/YYYY text	All 10/01/2026
Description	Free text	Includes the branch name in some rows (Rent (Marikina))
Supplier/Company Name, Address, TIN	Supplier details	Same supplier repeats: SHOPEE ×4, MAREGATO ×2, MERIDIEN ×2, with identical address/TIN
REMARKS	A reference number of mixed kinds	BD0#495979 (check), SI#0003089 (sales invoice), REF#5045632652067 (GCash)
Department	Which entity paid	ADT MARIKINA ×4, ADT DHCS INC ×4, ADT ANTIPOLLO ×2, ADT CAINTA ×1

CSV column	What it holds	Evidence
15 (header cell looks like an Excel artifact)	Services or Goods	Values are only Services / Goods
Classification	V or NV	5 V, 6 NV
Particulars	The expense category	Rent, Miscellaneous, Events, Dental Supplies, Office Supplies & equipment, Repairs
SOURCE OF FUND	How it was paid	CREDIT CARD ×7, BDO CHECK ×2, GCASH ×1, CASH ×1
Invoice Amount	Gross, VAT-inclusive	
VATable / Non-Vat / VAT-Exempt / VAT amount	The VAT split	Typed by hand

Problems in the sheet that the app should prevent:

- **The typed VAT numbers are inconsistent.** Of the 5 VAT rows, 4 differ from a plain $\div 1.12$ calculation. The worst: 84,984.64 $\rightarrow$ VATable 75,879.14 + VAT 9,104.91 = 84,984.05, which is 0.59 short of the invoice amount. The correct VAT is 9,105.50.
- **A non-VAT row has an amount in the VATable column** (the 118.00 Antipolo SHOPEE row has 9.00). Looks like a typing slip.
- **"ADT DHCS INC" is not a branch.** Its rows include Rent (Head Office) . It is the head office / company, and the app has no place for it.
- **The header says "September 2026" but every row is dated 10/01/2026.** See decision 6.
- **The date format is ambiguous** (10/01/2026 could be 10 January). It must be confirmed before any bulk import.
- **All 5 VAT rows have a supplier name and TIN** (5 of 5), while most non-VAT rows do not (cash purchases, Shopee). This gives a reliable validation rule: VAT requires a supplier and TIN; non-VAT does not.

2.3 The current importer will reject the new Day End layout (do this first)

I ran the same logic as `parse-csv.ts` against the file:

- The header row is found correctly.
- **All 9 real rows would fail**, with "Transaction type is required", because `transactionType` is required and the new report has no such column.
- The `Breakdown` rows and `Total` row would be treated as data (their Visit ID cells hold `Cash` , `300` and `3,965.00`) and add three more errors.

Proposed importer changes (`src/lib/cashflow/parse-csv.ts`):

1. Make Transaction Type, Visit Type, Month and Year optional in the file. Month and year are already derived from the date.
2. Derive the transaction type: `Reservation` when the Visit ID starts with `RES` or the procedure contains "reservation fee"; otherwise `Visit` .
3. Treat the table as ended at the first row whose Date does not parse, so the Breakdown and Total block is skipped.
4. Read the branch name from the title row (`MARIKINA DAY END REPORT`) and pre-select the matching branch in the import panel (still changeable). Today one branch dropdown applies to the whole file, so a wrong pick silently mislabels a day.

5. Use the Visit ID as the row id when it is unique within the file, and fall back to `visitId-n` only when it repeats. Today the id uses the row position, so re-importing a corrected file with rows in a different order creates duplicates (already noted in the project doc).
6. Create a payment row for each imported row, equal to its Amount Paid (see 3.3 below for why).

3. Adding a transaction

3.1 Discount (PWD / discounted price)

I read "PWD/Discounted field price" as: a way to record that a line was discounted, with PWD as one reason. If you meant something different, tell me.

Per line:

Field	Meaning
<code>list_price</code>	Price before discount
<code>discount_mode</code>	percent or amount (your "percentage / whole number")
<code>discount_value</code>	The number typed (a % or a peso amount)
<code>discount_amount</code>	The computed peso discount. Stored, because reports sum it
<code>discount_reason</code>	PWD, Senior citizen, Dental Network Member, Promo, Other

- **amountPaid keeps its current meaning** ("total due for this line", after discount). Dashboards, commission, net collection and the performance pages therefore need no change. The new columns are additive.
- Presets live in `constants.ts` (same convention as `PAYMENT_TYPES`): PWD 20%, Senior 20%, Dental Network Member 5%. The rates are examples to confirm (decision 2). Picking a preset fills the mode and value, which can still be edited.
- Rules: percent must be between 0 and 100; a peso discount cannot exceed the list price; a reason is required whenever the discount is above zero (discounts are where money leaks, so each one should be explainable).
- The final price is shown read-only under the line: $1,700.00 - 5\% = 1,615.00$.
- Rounding: discount is rounded to centavos once, then subtracted, so the line total is always an exact 2-decimal figure.
- Legacy rows: `list_price` is backfilled to the existing `amount_paid`, discount 0.

3.2 Multiple procedures per transaction

Today: one form submit creates one row with one procedure, one amount and a freshly generated Visit ID. Two procedures for the same patient on the same day become two unrelated "visits".

Options

	A. Header + lines (recommended)	B. Flat rows sharing an invoice number
Idea	New <code>invoices</code> table for visit-level data. Each procedure remains one row in <code>transactions</code> , linked by <code>invoice_id</code> .	No new table. N rows carry the same invoice number and Visit ID.

	A. Header + lines (recommended)	B. Flat rows sharing an invoice number
Dashboards, commission, performance	Unchanged (they read lines, as today)	Unchanged
Patient, date, invoice no., proof	Stored once on the invoice	Repeated on every row and can drift when one row is edited
Payments	Move to the invoice (one payment can cover several lines)	Stay per row; one card payment has to be split artificially
Printed invoice	Natural: one invoice, several lines	Needs grouping logic
Effort	Larger (migration + payments move)	Smaller, but more bugs later

Recommendation: A. The booklet invoice is the real-world unit: one patient, one number, several lines, one or more payments.

What changes under A:

- New `invoices` table: `id`, `invoice_number` (manual), `visit_date`, `patient_id`, `branch_id`, `transaction_type`, `visit_type`, `created_by_user_id`, `created_at`.
- `transactions` gains `invoice_id` and `line_number`. Existing rows each get a 1:1 invoice in a backfill script (same pattern as `db:backfill-payments`).
- `payments` gains `invoice_id`; the backfill fills it for every existing payment, so balance logic reads only one place afterwards. Balance due = sum of the invoice's line totals – sum of its payments.
- Merchant fee and withholding tax belong to a payment, not a line. They are entered once with the payment and **allocated across the lines in proportion to each line's total** (the last line absorbs the rounding), so line-level net collection and per-dentist numbers still add up exactly.
- Dentist is chosen per line (a "same dentist for all lines" shortcut), because dentist and commission are per line today and two dentists on one visit is real (`TEOPE/BANDAYREL` exists in the dentist list).
- Visit ID, transaction number and new/returning are decided once per invoice.
- The transactions table keeps showing one row per line, with the same invoice number and balance pill repeated on lines of one invoice.

Files touched under A: `db/schema.ts`, `cashflow/schema.ts`, `db/transactions.ts` (new `createInvoiceWithLines`), `actions.ts`, `transaction-form.tsx` (header + line editor), `transactions-table.tsx`, `transaction-detail.tsx`, `invoice-view.tsx`, `payments.ts`, `payment-actions.ts`, `payments-manager.tsx`, plus one SQL migration and one backfill script.

Commission with discounts is a policy question, not a technical one (decision 1). By default, commission is calculated on net collection, which is already after discount.

3.3 Latent issue that this feature should fix: new transactions have no payment

Right now neither the Add Transaction form nor the CSV import creates a payment row. Only the backfill script and the manual Payments panel do. The result is that **every newly added or imported transaction shows "PX due" until someone records a payment on its detail page**. The Day End import would show all nine rows as unpaid.

Proposed: the new form gets a **Payment** section ("Amount received now", payment type, reference no., proof file) that creates the first payment together with the invoice. The CSV importer creates one payment per row equal to Amount Paid, as in point 6 of section 2.3. The field currently labelled "Amount paid" in the form is really the total due, so it becomes **Total due** (calculated from the lines) to stop clashing with the Day End report's meaning of "Amount Paid" (money received).

Also worth fixing while here: `addPaymentAction` never checks the payment against the balance, so an overpayment is accepted (already listed in `ARCHITECTURE_GAP_ANALYSIS.md`).

3.4 VAT / non-VAT — the efficient approach

Today `vatExclusive` and `vatAmount` are typed values that the manual form does not even collect. The expenses sheet shows why typed VAT drifts.

Solution:

1. One enum per line: `vat_type = vat | non_vat | vat_exempt` (text column validated by Zod, like the other category fields, so adding a value later needs no database enum change).
2. One helper, `computeVat(gross, vatType)`, in a new `src/lib/cashflow/vat.ts`:
 - `vat`: `base = round(gross ÷ 1.12)`, `VAT = gross - base`. This guarantees `base + VAT = gross` exactly, which the sheet fails in 4 of 5 rows.
 - `non_vat` / `vat_exempt`: `base = gross`, `VAT = 0`.
3. `vatExclusive` and `vatAmount` stay as stored, **derived** columns, so existing reports and historical rows are untouched.
4. **Default per procedure**: add `procedures.default_vat_type` (and optionally `default_price`). Staff then rarely touch the VAT field. For this clinic it is non-VAT almost always, so it is a small override, not a decision on every line.
5. The same helper and the same three values serve the **expenses** form, so there is one VAT implementation in the app.
6. The 12% rate is one constant (`VAT_RATE`), not repeated.

PWD and senior discounts normally come with their own VAT treatment under Philippine rules. I would not hard-code it; the preset can optionally set the VAT type, which the clinic's accountant should confirm (decision 2).

3.5 Patients: do we need a patients table?

Yes. Reasons:

- Typeahead needs a source of people, and "new vs returning" needs an identity. Today both are guessed from the name string (`dashboard-metrics.ts` lowercases and compares `patientName`).
- A typo (`Pasco, Kairah Trish` vs `Pasco, Kairah`) splits one patient in two; two different people with the same name merge into one.
- Reservation-fee rows (`RES...`) carry a patient name before any visit exists. The person should be the same record when they later visit.
- It is where PWD/senior ID, contact number and notes naturally live.

Cheaper alternative: run the typeahead off `SELECT DISTINCT patient_name FROM transactions`. It works for suggestions, but cannot merge duplicates or hold any patient detail. I would only choose it if you want to avoid the migration.

Design (follows the pattern already used for `dentist_user_id` and `procedure_id`):

- `patients`: `id`, `full_name` (as `Last, First`), `name_key` (lowercased, trimmed, collapsed spaces, indexed), `contact_number` (optional), `notes` (optional), `created_at`.

- `transactions.patient_name` **stays** (display text, and CSV rows keep working).
`transactions.patient_id` is added, nullable.
- On save and on import, link by **exact normalized match only**, never fuzzy matching, and create the patient if there is none. The same rule already protects commission links.
- Backfill script: distinct normalized names → patients → link existing transactions. It reports names that look like near-duplicates for a human to review; it does not merge them.

Typeahead:

- New `PatientCombobox`, built on the existing `ProcedureCombobox` (keyboard navigation, listbox roles and the "+ Add as new" row are already done there).
- The procedure list is small and passed as a prop; patients will grow into the thousands, so search is **server-side**: a Server Action called after about 200 ms of no typing, prefix match on `name_key` first (it suits `Last, First`), then contains-match, limited to 8 results.
- Choosing a result shows a small card (last visit date, last dentist, new/returning). Typing a name with no match offers "+ New patient". If the normalized name matches an existing patient, it warns instead of silently creating a duplicate.

New vs returning:

- Computed on the server when the invoice is saved: **Returning** if the patient has at least one earlier `visit`-type transaction, otherwise **New**.
- Reservations do not count as a visit. A zero-amount follow-up does count.
- The result is stored in the existing `visit_type` column, so the dashboard and CSV continue to work. A small "override" control handles patients who were seen before the system had their data.
- The Day End report has no Visit Type column, so this also fills a gap in imported rows once patients exist.

3.6 Invoice number — manual

- The field becomes a normal editable text input. `generateInvoiceNumber` is no longer called for new transactions. The internal `transactionNumber` (`MMDDYY-####`) stays automatic as the app's own reference. The printed invoice currently prefers `transactionNumber` over `invoiceNumber` (`invoice-view.tsx`); that order flips so the booklet number is shown first, falling back to `transactionNumber` only when none was entered.
- It is **optional by default**, because 8 of 9 rows in the Day End report have none. A missing invoice number gets a gentle warning ("No booklet number yet") and a "Missing invoice no." filter on the transactions list, so gaps can be chased at day end.
- Duplicate protection: unique per branch when not blank (booklets are normally per branch; decision 3). A duplicate shows "4543 was already used on Oct 3 for Bo, Arwen" and blocks the save.
- A hint under the field shows the last number used in that branch (for example "last used: 4543") without filling it in, since the booklet is the source of truth.
- It lives on the `invoices` row; a copy stays on `transactions.invoice_number` for search and legacy rows.

3.7 Proof of payment upload

I read "import file for proof of payment" as attaching a receipt image or PDF to a payment. If you meant importing a spreadsheet, tell me.

- **Two separate things:** a short **Reference no.** field (GCash ref, GHIL approval no., check no.), which today is typed into Remarks, and the **uploaded file**. Reference no. is stored on the payment and searchable.
- **Where files go:** private object storage, not the database. Database storage is the wrong place for receipt photos (it grows quickly and is the most expensive storage), and a public URL for patient and bank details is not acceptable.
- **Table payment_proofs:** `id`, `payment_id`, `storage_key`, `file_name`, `mime_type`, `size_bytes`, `uploaded_by_user_id`, `created_at`.
- **Upload path:** straight from the browser to storage using a short-lived signed URL. Next.js Server Actions default to a 1 MB body limit, so sending a phone photo through one would fail.
- **Viewing:** an authenticated route that checks the session and returns a short-lived signed link. Every signed-in role can view; only admins can delete (matching payment deletion).
- **Limits:** JPG, PNG, WebP or PDF, up to 5 MB, up to 3 files per payment. The server checks the file's actual type, not just its extension. Images are shrunk in the browser first (longest side about 1,600 px) so a 4 MB photo becomes a few hundred KB.
- **Reuse:** the same uploader component serves the expenses form. That makes the "storage of receipts" item noted earlier in the project (deferred then) almost free to turn on.
- **New dependency:** depends on the host. `@vercel/blob` if the app is on Vercel, otherwise `@aws-sdk/client-s3` with `presigner` for S3 or Cloudflare R2 (decision 4). As usual, you run the install; I only edit files.

3.8 The new Add Transaction form

Invoice

Invoice no. [4544] last used in Marikina: 4543 Date [2026-10-03]

Patient [Zamora, Ca▼] -> Returning · last visit Sep 21 · Dr. TEOPE

Branch [Marikina ▼] Transaction type [Visit ▼]

Procedures [+ Add procedure]

#	Procedure	Dentist	List price	Discount	VAT	Total
1	[Sanitation ▼]	[TEOPE ▼]	[500.00]	[PWD ▼] [20 %▼]	[Non-VAT▼]	400.00 [x]
2	[Ortho Adj ▼]	[TEOPE ▼]	[800.00]	[none]	[Non-VAT▼]	800.00 [x]
Subtotal 1,300.00 · Discounts -100.00 · VAT 0.00						
Total due 1,200.00						

Payment

Type [GoTyme ▼] Received now [1,200.00] Reference no. [0045705066371]

Merchant fee [0.00] Withholding tax [0.00] Proof of payment [Choose file]

Remarks []

Card fees: for POS credit the form can suggest the fee and withholding tax from the rates seen in the sample (2.8% and 0.5%), shown as editable suggestions. I would not apply them silently (decision 9).

4. Add expenses

4.1 CSV column → app field

CSV column	App field	Change
Date	<code>date</code>	Exists
Description	<code>description</code>	Exists
Supplier/Company Name	<code>supplier_id</code> → <code>suppliers.name</code>	New table , typeahead, optional
Address	<code>suppliers.address</code>	New (auto-filled from the supplier)
TIN	<code>suppliers.tin</code>	New (auto-filled from the supplier)
REMARKS	<code>reference_no</code> plus <code>remarks</code>	New. The sheet mixes check no., sales invoice no. and GCash ref
Department	<code>branch_id</code>	Exists, but needs a Head Office entry (decision 5)
15 column (Services/Goods)	<code>nature = services / goods</code>	New
Classification (V/NV)	<code>vat_type = vat / non_vat / vat_exempt</code>	New, shared with section 3.4
Particulars	<code>category</code>	Exists; the list changes (4.3)
SOURCE OF FUND	<code>payment_method</code> , relabelled Source of fund	Exists, but needs its own list (4.3)
Invoice Amount	<code>amount</code>	Exists (gross, VAT-inclusive)
VATable / Non-Vat / VAT-Exempt / VAT amount	<code>vatable_amount</code> , <code>vat_amount</code> stored; the other two derived	New

Only two VAT columns need storing. The sheet's Non-Vat and VAT-Exempt columns can be produced by a simple CASE on `vat_type` when exporting, so there is nothing to keep in sync.

Terminology: I would label the category field **Particulars** to match the clinic's own word, with "category" only in code.

4.2 New expense fields and suppliers

- `expenses`: add `supplier_id` (nullable FK), `reference_no`, `remarks`, `nature`, `vat_type`, `vatable_amount`, `vat_amount`, and later `receipt` through the shared uploader.
- `suppliers`: `id`, `name`, `address`, `tin`, `created_at`. The CSV repeats suppliers with identical address and TIN, so one typeahead removes retyping. Same combobox pattern as patients (small list, so it can be passed as a prop).
- Supplier details are linked, not copied. Editing a supplier's TIN would change the history shown for past expenses; edits are therefore admin-only. If the accountant needs the TIN exactly as it was on the day of the invoice, we can store a snapshot instead (decision 10 includes this).

4.3 Lists that change

- Particulars**: replace the current list (Rent, Utilities, Salaries, Dental supplies, Equipment, Marketing, Maintenance & repairs, Insurance, Taxes & licenses, Other) with the clinic's own: Rent, Dental Supplies, Office Supplies & equipment, Repairs, Events, Miscellaneous. Keep Utilities,

Salaries, and Taxes & licenses as likely, because the sample is one day. The file only shows six values, so please send the full list. Any test expenses already saved with the old labels should be few; they can be remapped with one UPDATE.

- **Source of fund:** Cash, GCash, Credit Card, BDO Check (plus Bank Transfer if used). The form currently reuses `PAYMENT_TYPES`, which lists HMO providers such as Maxicare and does not make sense for money going out.

4.4 Form and validation

Sections: **Basics** (date, department, particulars, description, services/goods); **Supplier** (typeahead, address and TIN shown read-only once chosen, "+ New supplier" inline); **Invoice** (reference no., VAT classification, invoice amount, with a read-only breakdown "VATable 75,879.14 · VAT 9,105.50"); **Payment** (source of fund, optional receipt, remarks).

Rules drawn from the sample:

- Amount greater than 0.
- `vat` requires a supplier and a TIN (true for 5 of 5 VAT rows in the sample). `non_vat` does not.
- VAT is never typed: it is calculated, so `vatable + VAT = invoice amount` always holds.
- Cash purchases with no supplier (`Screw for cabiner`) stay valid.
- Warn on a likely duplicate: same supplier, reference no. and amount.

4.5 Optional: expense CSV import

The clinic keeps this sheet monthly, so an importer will probably be asked for. It is not in your request, so I would only build it after the form. Points the parser must handle: header row is not the first row; descriptions with trailing line breaks; thousands separators; the unlabeled `15` header; `MM/DD/YYYY` dates, which need explicit confirmation first and a preview before anything is saved; and the data slips above (a non-VAT row with a VATable amount) shown as row warnings, not silent fixes.

5. Data model changes (consolidated)

Table	Change
<code>patients</code>	New
<code>invoices</code>	New
<code>payment_proofs</code>	New
<code>suppliers</code>	New
<code>transactions</code>	Add <code>invoice_id</code> , <code>line_number</code> , <code>patient_id</code> , <code>list_price</code> , <code>discount_mode</code> , <code>discount_value</code> , <code>discount_amount</code> , <code>discount_reason</code> , <code>vat_type</code>
<code>payments</code>	Add <code>invoice_id</code> (backfilled), <code>reference_no</code>
<code>procedures</code>	Add <code>default_vat_type</code> , optionally <code>default_price</code>
<code>expenses</code>	Add <code>supplier_id</code> , <code>reference_no</code> , <code>remarks</code> , <code>nature</code> , <code>vat_type</code> , <code>vatable_amount</code> , <code>vat_amount</code>
<code>branches</code>	Add a Head Office row (decision 5)

How it ships, following the repo's own rules:

- One idempotent SQL file per phase in `scripts/sql/` (`IF NOT EXISTS` , constraint adds wrapped for re-runs), because `yarn db:push` still hits the drizzle-kit Postgres 17 bug. You run each file in the Neon SQL editor.
- Backfill scripts in `scripts/` for patients, invoices and payments, safe to re-run, like the existing ones.
- Everything is additive. Nothing is dropped or renamed, and old rows keep working through defaults.
- I do not run installs, `yarn dev` or any database command. Checking is `tsc --noEmit` and `eslint`.

6. Decisions I need from you

1. **Commission on discounted sales.** Should a dentist's commission be on the list price or on the discounted price? The default (net collection, after discount) lowers commission on every PWD or network-member sale. Also, who bears a PWD discount, the clinic or the dentist?
2. **PWD / Senior rates, VAT treatment and ID number.** Please confirm with the accountant: the rate (20% is the commonly used figure), whether the discount changes the VAT type, and whether the invoice must record the cardholder's ID number (I would add it as an optional field on the patient).
3. **Invoice number.** Optional with a warning (my recommendation) or required? Are booklets numbered per branch, so uniqueness is per branch?
4. **Where is the app hosted?** This decides the file storage choice (Vercel Blob, S3 or R2). It blocks only the proof-of-payment feature.
5. **Head Office.** Should "ADT DHCS INC" be added as a branch-like entry for expenses? If so, it will also appear in the branch picker for transactions unless I add a flag to hide it there.
6. **Expense dates.** The header says September 2026 and every row is dated 10/01/2026. Is the format month/day, and are September bills paid on October 1 meant to count in September or October? The dashboard buckets by the date typed.
7. **Visit ID format.** Should the app generate the clinic's own `202610-003-001` (looks like year-month, day, sequence) and `RES20261003-010` instead of `20260921-001` , so the app and the paper report match?
8. **Reservation fees** count toward the Day End total (600.00 of 3,965.00). Confirm they should also count toward net collection on the dashboard, which they do today.
9. **Card fee suggestions.** Confirm 2.8% merchant fee and 0.5% withholding tax for GHL credit, and which payment types they apply to, before I suggest them in the form.
10. **Expense categories and supplier snapshot.** Please send the full Particulars list, and say whether the accountant needs supplier TIN/address frozen as of each invoice.

Decisions 1, 3, 4 and 5 change the design. The rest adjust defaults.

7. Order of work

Phase	Work	Needs	Size
0	Fix the importer for the Day End layout (2.3), including the payment rows	Nothing	S

Phase	Work	Needs	Size
1	Shared foundations: <code>vat.ts</code> , discount helper, new constants, SQL migration 1 (additive columns, <code>patients</code> , <code>suppliers</code>)	Decisions 2, 10	S
2	Patients: backfill script, <code>PatientCombobox</code> , new/returning	Phase 1	M
3	Expenses form v2 plus suppliers	Phase 1, decisions 5, 6	M
4	Invoices, line editor, discounts, VAT per line, manual invoice number, first payment in the form	Phases 1-2, decisions 1, 3, 7	L
5	Proof of payment uploader (then reuse on expenses)	Phase 4, decision 4	M
6	Optional: expense CSV import, "Discounts given" card on the dashboard	Phases 3, 4	S-M

Phase 0 is independent and worth doing right away, because daily Day End files cannot be imported today. Phases 2 and 3 are independent of each other. Phase 4 is the biggest and the one where existing data is restructured, so I would do it after the backfills for patients and invoices have run cleanly on your data.

8. Risks

- **Phase 4 touches the payment flow.** Moving balance logic to the invoice is the riskiest change. Mitigation: backfill every existing payment with its `invoice_id` first, keep the old column until the new logic is verified, and check balances before and after the backfill for a sample of transactions.
- **Patient merge errors.** Exact-match linking will merge two different people who share a name, and will not merge typos. The backfill report lists near-duplicates for review, and the same limitation already exists today.
- **Typed numbers versus computed numbers.** Computed VAT will not always equal what the old sheet shows (centavo differences). That is intended, but the accountant should know before the first month-end.
- **Commission changes.** Any change to how discounts feed commission alters dentists' pay; please settle decision 1 first.
- **Uploaded files contain personal and bank details.** Private storage, signed links and admin-only deletion are the minimum. Receipts should not be logged or put in URLs.
- **Existing data volume.** Only a few transactions exist at the moment, so migrations are cheap now. They get more expensive once the clinic has a few months of real data.

9. How each phase will be checked

- `tsc --noEmit` and `eslint` clean after every phase.
- Pure functions (`computeVat` , discount, fee allocation, new/returning rule, CSV derivations) kept in `src/lib/cashflow/` , with these figures from your own files as acceptance examples:

Case	Input	Expected
Percent discount	List 1,700.00, 5%	Discount 85.00, total 1,615.00

Case	Input	Expected
Card deductions on that sale	1,615.00	Fee 45.22, withholding 8.08, net 1,561.70
Non-VAT sale	1,300.00, non_vat	Base 1,300.00, VAT 0.00
VAT expense	84,984.64, vat	Base 75,879.14, VAT 9,105.50
VAT expense	4,850.00, vat	Base 4,330.36, VAT 519.64
Reservation	Visit ID RES20261003-010 , GCash 150.00	Type Reservation, no dentist, net 150.00
No-charge follow-up	0.00	Accepted
Day End file	The October 3 file	9 transactions, 0 errors, total 3,965.00, summary block skipped
Fee split across lines	Merchant fee 45.22 on an invoice with lines 1,000.00 and 615.00	28.00 and 17.22, which add back to 45.22

- Each SQL file is idempotent, so you can run it twice in Neon without harm, and each backfill prints counts before and after.